Jujutsu workspaces: Scale and Scale-code

Status: applied. The repository is colocated with Jujutsu. A second workspace, `Scale-code`, is a sparse checkout of the Scale-owned source and documentation. It excludes `components/nu-engine`, so the F# language server sees only Scale projects. The main `Scale` workspace remains the full checkout and the home of Nu work.

Why two workspaces

Serena launches `fsautocomplete` with `--project-graph-enabled` and `workspacePath` set to the repository root. With the full checkout, that graph contains 37 projects: the Scale projects plus 21 vendored Nu projects. The Nu projects are not built, so the graph reports `Overall Build: Failure` while looking for `Nu.Math.dll` and `Nu.dll`. The Scale projects are then missing from `LoadedProjects`, and symbol calls fail even though `just lsp-check` passes.

`.ignore` and Serena's `ignored_paths` do not help: the F# server has its own project discovery. The durable fix is to give Serena a root where the Nu projects are not present. A Jujutsu workspace is a second working copy over the same repository history, so it can be sparse without copying or rewriting the main checkout.

Workspace	Path	Contents	Use
`Scale`	`/home/shalok/Projects/Scale`	Full checkout, including `components/nu-engine`.	Nu work, engine builds, host integration, and engine-adjacent review.
`Scale-code`	`/home/shalok/Projects/Scale-code`	Sparse checkout of the Scale-owned paths; `components/nu-engine` is absent.	Scale source, tests, docs, and the Serena/F# LSP.

The two workspaces share one Jujutsu repository, one operation log, and one Git object store. They do not share working-copy files. A change made in one is a commit that the other can see in `jj log`; it is not automatically present in the other working copy.

Prerequisites

1. `jj` 0.46.0 or newer.
2. A colocated repository: `/home/shalok/Projects/Scale/.jj` exists beside `.git`.
3. The main checkout is clean enough to snapshot. Jujutsu snapshots the current working-copy commit before creating a workspace.

Create the workspace

The workspace is created empty so the vendored Nu tree is never materialized. `-r @` makes the current working-copy commit the parent, so the new workspace starts with the current source and documentation, not an older commit.

```
cd /home/shalok/Projects/Scale

jj workspace add \
  --name scale-code \
  -r @ \
  --sparse-patterns empty \
  /home/shalok/Projects/Scale-code
```

Then materialize only the Scale-owned paths. This list deliberately names each component instead of adding `components/`, because adding the parent would also add `components/nu-engine`.

```
cd /home/shalok/Projects/Scale-code
```

```
jj sparse set --clear \
  --add adapters \
  --add bases \
  --add development \
  --add docs \
  --add Notes \
  --add projects \
  --add tools \
  --add .config \
  --add .github \
  --add components/adapter-c \
  --add components/cli \
  --add components/core \
  --add components/demo \
  --add components/ipc \
  --add components/kernel \
  --add components/nu-host \
  --add components/prelude \
  --add components/typing \
  --add components/verification \
  --add .editorconfig \
  --add .fantomasignore \
  --add .gitignore \
  --add .ignore \
  --add AGENTS.md \
  --add Directory.Packages.props \
  --add global.json \
  --add justfile \
  --add LICENSE \
  --add LICENSE-docs \
  --add README.typ \
  --add Scale.slnx \
  --add workspace.edn
```

`--sparse-patterns empty` is the Jujutsu 0.46 feature this setup relies on. This version does not expose sparse exclusions, so the Scale paths are listed explicitly. If a later Jujutsu version supports exclusions, the same workspace can be expressed as "everything except `components/nu-engine`".

Verify it

Run these from `/home/shalok/Projects/Scale-code`.

```
jj workspace list
jj sparse list
jj status
test ! -e components/nu-engine && echo "Nu is absent"
just docs-check
```

`jj workspace list` must show `default` at the main checkout and `scale-code` at the new checkout. `jj sparse list` must not contain `components/nu-engine`. `jj status` must report no changes immediately after creation. `just docs-check` passes in both workspaces: when the Nu project is absent, it skips the external Nu package-policy check instead of failing on a missing external node.

The Serena project root must be `/home/shalok/Projects/Scale-code`, not the main checkout. Activate it once:

```
serena activate_project /home/shalok/Projects/Scale-code
```

or, from an agent session, use the Serena `activate_project` tool with that path. Serena creates the workspace-local `.serena` directory and starts its own F# server. Then confirm the fix on a real project file:

```
get_symbols_overview components/prelude/scheduler.fs
```

This returns the `Scheduler` module and its fields. That is the check `just lsp-check` does not make: `lsp-check` uses a `.fsx` and can pass while the Scale project graph is still broken.

Daily use

Scale work. Open `/home/shalok/Projects/Scale-code`, run SageFs with that directory, and use the normal commands. The F# server sees the Scale projects only. The Nu engine is intentionally absent.

Nu work. Open `/home/shalok/Projects/Scale` and use the main workspace. The Nu plan and engine work belong here because the engine source and its project graph are present.

One workspace per task. Use the workspace as the isolation boundary: do not edit the same logical change in both working copies. Finish or describe the change in one, then move it through the shared commit graph.

Everyday Jujutsu

The everyday commands in a workspace are:

```
jj status                 # working-copy changes
jj diff                   # the current change
jj log                    # history, including both workspaces
jj show <change>          # inspect a change without checking it out
jj describe -m "message"  # describe the current working-copy change
jj new                    # start a new change on top of the current one
jj undo                   # undo the last operation
jj op log                 # the operation log
```

The working copy is itself a commit. `jj describe` changes its message; `jj new` starts the next change. A bookmark is a named pointer, not a branch:

```
jj bookmark create <name> -r <change>
jj bookmark set <name> -r <change>
jj bookmark list
```

In a colocated repository, `git status`, `git diff`, and `git log` still work, but Jujutsu owns the working copy. Do not use `git commit`, `git checkout`, or `git reset` to move work; use `jj describe`, `jj new`, `jj edit`, and `jj undo`. Do not push unless the user asks for it.

Moving work between workspaces

Both workspaces share the operation log. Use `jj log` to see change ids from either working copy, and `jj show <change>` to inspect a change without checking it out.

For a handoff:

```
# In the source workspace, name the change.
jj describe -r <change> -m "describe the change"
```

```
# In the destination workspace, move the change onto the current commit.
jj rebase -s <change> -d @
```

If the change has descendants, `-s` moves the whole subtree; `-b` moves only the branch containing the change. Resolve any conflict in the destination working copy, then verify there. A bookmark is useful when a change must be named across sessions:

```
jj bookmark create <name> -r <change>
jj bookmark list
```

Jujutsu automatically rebases descendants when a parent commit is rewritten. If the main workspace rewrites a commit that `Scale-code` is based on, the `Scale-code` working copy may move or report a conflict. That is normal; resolve it in `Scale-code` and re-run the relevant SageFs checks. If the workspace is reported as stale after another workspace moved the shared history, run:

```
jj workspace update-stale
```

Do not copy files between workspaces with `cp`, `rsync`, or a Git checkout. That creates two unrelated changes and defeats the shared history.

Sparse maintenance

Add a new Scale path when a new top-level component or project appears:

```
jj sparse set --add components/new-component
```

Remove a path from this workspace:

```
jj sparse set --remove components/new-component
```

Replace the whole set with the Scale-only list again:

```
jj sparse set --clear --add ...   # the same list as above
```

`jj sparse reset` restores the full checkout, including `components/nu-engine`. Do not run it in `Scale-code` unless the intent is to turn that workspace into a full checkout. The main `Scale` workspace is already the full checkout.

Safety and limits

1. Do not run `jj workspace forget scale-code` while it holds unmerged work. Forgetting is a workspace-registration operation, not a merge.
2. The workspace is created as a Git worktree because the repository is colocated. Manage it with Jujutsu; do not remove it with `git worktree remove` or switch it with `git checkout`.
3. Sparse checkout changes what is present in the working copy, not what is in the commit or the repository. A file absent from `Scale-code` is still in the commit and can be restored with `jj sparse set`.
4. The Nu license and the external-node boundary are unchanged by the workspace. `components/nu-engine` remains external and noncommercial; `Scale-code` does not relicense it.
5. The workspace list is repository state, not a Polylith node. `workspace.edn`, `Scale.slnx`, and the dependency graph are unaffected.

Failure mode and prevention

The full `Scale` checkout is not a reliable F# symbol workspace. FsAutoComplete starts with automatic workspace initialization, scans every `.fsproj` under the root, and also discovers the external Nu projects. A failed Nu design-time build leaves `LoadedProjects` without the Scale projects, so a document-symbol request for a file that exists reports `Couldn't find ... in LoadedProjects`.

Server health is not project load. `just lsp-check` therefore also indexes `components/prelude/evaluator/evaluator.fs` and fails with an explicit message when that file cannot be loaded. If the checkout contains `components/nu-engine`, the message points to the sparse `Scale-code` workspace.

The operating rule follows from the Polylith layout:

1. full `Scale`: Nu work, engine builds, host integration;
2. sparse `Scale-code`: Scale-only F# navigation, editing, and SageFs work;
3. `components/nu-engine`: the external node, never part of the Scale project graph and never a place for Scale source changes;
4. new Scale code: the owning node under `components/`, `bases/`, or `projects/`.

The full rationale and the staged prevention work are in `Notes/lsp-workspace-plan.md`.

Current state

Created on 2026-11-04 with Jujutsu 0.46.0:

1. `default` workspace: `/home/shalok/Projects/Scale`.
2. `scale-code` workspace: `/home/shalok/Projects/Scale-code`.
3. `Scale-code` contains the Scale-only path set; `components/nu-engine` is absent.
4. Serena symbol lookup on `components/prelude/scheduler.fs` succeeds in `Scale-code`.